

PostgreSQL Backup Strategies

Table of Contents

1. [Learning Objectives](#)
 2. [Why Backup Strategy Matters](#)
 3. [RPO and RTO Defined](#)
 4. [Logical vs Physical Backups](#)
 5. [Backup Types: Full, Differential, Incremental](#)
 6. [Backup Methods Comparison](#)
 7. [Choosing the Right Strategy](#)
 8. [Multi-Tier Backup Architecture](#)
 9. [Common Mistakes](#)
 10. [Best Practices](#)
 11. [Interview Questions](#)
 12. [Exercises and Solutions](#)
-

Learning Objectives

By the end of this module, you will be able to:

- Define RPO and RTO and set realistic targets for different applications
 - Choose between logical and physical backup methods
 - Design a multi-tier backup strategy (full + incremental + WAL)
 - Understand the trade-offs between backup types
 - Create a backup policy document for an organization
-

Why Backup Strategy Matters

WHAT BACKUPS PROTECT AGAINST:

Hardware failure	→ Disk crash, RAID failure
Software corruption	→ PostgreSQL bug, OS crash
Human error	→ DROP TABLE, wrong UPDATE
Security breach	→ Ransomware, malicious deletion
Datacenter disaster	→ Fire, flood, power loss
Application bugs	→ Logic errors corrupting data

WHAT BACKUPS DO NOT REPLACE:

High Availability (Patroni/Streaming replication)

- HA protects against server failure (seconds recovery)
- Backup protects against data loss/corruption (minutes/hours recovery)
- YOU NEED BOTH!

RPO and RTO Defined

RTO (Recovery Time Objective):

"How long can the application be unavailable?"

Time from incident declaration to system restored.

Examples:

- E-commerce: RTO = 15 minutes
- Internal HR system: RTO = 4 hours
- Archive database: RTO = 24 hours

RPO (Recovery Point Objective):

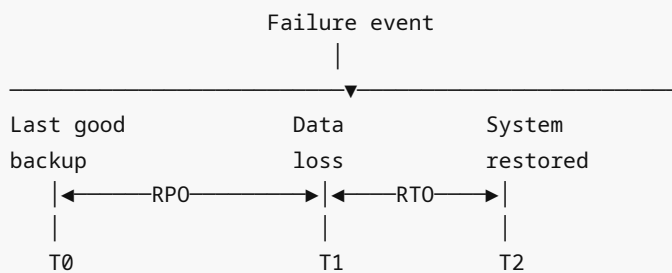
"How much data can we lose?"

Maximum age of data at time of recovery.

Examples:

- Financial system: RPO = 0 (no data loss)
- Order system: RPO = 5 minutes
- Analytics: RPO = 1 hour

VISUAL:



RPO/RTO vs. Backup Strategy

RPO	RTO	Recommended Strategy
0 (zero loss)	< 60s	Synchronous replication + Patroni
< 5 min	< 15 min	Streaming replication + WAL archiving + Patroni
< 1 hour	< 2 hours	WAL archiving with 5-min WAL rotation
< 24 hours	< 4 hours	Daily physical backup + WAL
< 7 days	< 8 hours	Weekly full + daily pg_dump

Logical vs Physical Backups

Logical Backups (pg_dump, pg_dumpall)

Logical backup: Exports data as SQL statements or CSV

- SELECT data from tables
- Export as INSERT statements or custom format

Advantages:

- ✓ Cross-version compatible (dump PG13, restore to PG16)
- ✓ Cross-platform (dump on Linux, restore on Windows)
- ✓ Selective restore (single table, single schema)
- ✓ Human-readable (plain SQL format)
- ✓ Small size (no bloat, no dead tuples)

Disadvantages:

- ✗ Slow for large databases (serialized read + write)
- ✗ No point-in-time recovery within dump (snapshot at backup time)
- ✗ Inconsistent if not done in transaction (use --serializable-deferrable)
- ✗ Does not backup roles, tablespaces (need pg_dumpall for those)
- ✗ RTO is high: restore by executing all SQL

Physical Backups (pg_basebackup, pgBackRest)

Physical backup: Copies raw data files (binary blocks)

- Copy pg_data directory
- Copy WAL files

Advantages:

- ✓ Fast restore (copy files back vs execute SQL)
- ✓ Point-in-time recovery (combine with WAL archiving)
- ✓ Consistent by design (based on WAL checkpoint)
- ✓ Full cluster backup (all databases, global objects)
- ✓ Works for large databases at scale

Disadvantages:

- ✗ Same major PostgreSQL version required for restore
- ✗ Cannot restore single table (need logical extract after restore)
- ✗ Larger backup size (includes bloat, dead tuples)
- ✗ More complex setup (WAL archiving for PITR)

Comparison Table

Feature	Logical (pg_dump)	Physical (pg_basebackup)
Speed (backup)	Slow	Fast
Speed (restore)	Slow	Fast
Size	Smaller	Larger
PITR possible	No	Yes (+ WAL archiving)
Selective restore	Yes (table/schema)	No (full cluster)
Cross-version	Yes	No
All databases	pg_dumpall	Yes
Complexity	Low	Medium

Backup Types: Full, Differential, Incremental

Full Backup

Complete copy of all data at a point in time.

pg_dump for logical:

```
pg_dump -Fc mydb > mydb.dump
```

pg_basebackup for physical:

```
pg_basebackup -D /backup/full/20240115 --wal-method=stream
```

pgBackRest full:

```
pgbackrest --stanza=mydb backup --type=full
```

Pros: Self-contained, simplest restore

Cons: Time-consuming, large storage per backup

Differential Backup

All data changed since the LAST FULL backup.

pgBackRest differential:

```
pgbackrest --stanza=mydb backup --type=diff
```

Restore = Latest full + Latest differential

Example schedule:

```
Sunday:    Full backup    (100% baseline)
Monday:    Diff backup    (changes since Sunday)
Tuesday:    Diff backup    (changes since Sunday, larger)
...
Saturday:  Diff backup    (6 days of changes)
Sunday:    New Full backup
```

Pros: Faster restore than incremental (only 2 backups needed)

Cons: Grows through the week; larger than incremental

Incremental Backup

Only data changed since the LAST backup (full or incremental).

pgBackRest incremental:

```
pgbackrest --stanza=mydb backup --type=incr
```

Restore = Latest full + All incrementals in sequence

Example schedule:

```
Sunday:    Full backup    (100%)
Monday:    Incr backup    (1 day of changes)
```

Tuesday: Incr backup (1 day of changes)
Wednesday: Incr backup (1 day of changes)
...

Pros: Smallest individual backup size
Cons: Restore requires full + all incrementals (more complex, slower)

Backup Methods Comparison

Method	Type	Best For	Limitations
pg_dump	Logical, selective	Single DB exports, migrations	No PITR, slow restore
pg_dumpall	Logical, full	Small DBs, global objects	No PITR, very slow restore
pg_basebackup	Physical, full	Initial replica setup, simple PITR	Full copy every time
pgBackRest	Physical, all types	Production, large DBs, full PITR	Setup complexity
WAL-G	Physical, all types	Cloud-native, S3-based	Setup complexity
COPY TO	Logical, table-level	Specific table exports	Manual, no metadata

Choosing the Right Strategy

Decision Tree:

```
START
├── Database size < 50 GB?
│   ├── YES → pg_dump (simple, adequate)
│   └── NO  → Physical backup required
├── Need point-in-time recovery?
│   ├── YES → WAL archiving required (physical + WAL)
│   └── NO  → pg_dump or pg_basebackup sufficient
├── Production/mission-critical?
│   ├── YES → pgBackRest (full-featured, retention management)
│   └── NO  → pg_basebackup or pg_dump
├── Cloud/S3 storage?
│   ├── YES → pgBackRest S3 or WAL-G
│   └── NO  → pgBackRest local/SSH
└── RPO < 1 hour?
    ├── YES → WAL archiving (continuous WAL shipping)
    └── NO  → pg_dump on schedule
```

Multi-Tier Backup Architecture

PRODUCTION BACKUP ARCHITECTURE:

Level 1: Streaming Replication (HA, not backup)

Primary —WAL—> Standby

RT0: 30-60 seconds

RPO: 0 (sync) / seconds (async)

Protects against: server failure

Level 2: WAL Archiving (PITR)

Primary —archive_command—> S3/NFS

WAL archived every 5 minutes or on segment fill (16MB)

RPO: 5 minutes

Protects against: accidental data deletion, logical errors

Level 3: Incremental Physical Backups (Recovery)

pgBackRest: weekly full, daily incremental

Stored: Local + S3 (cross-region copy)

RPO: To most recent WAL

RT0: 30-90 minutes for 500GB database

Protects against: storage failure, backup corruption

Level 4: Long-term Retention

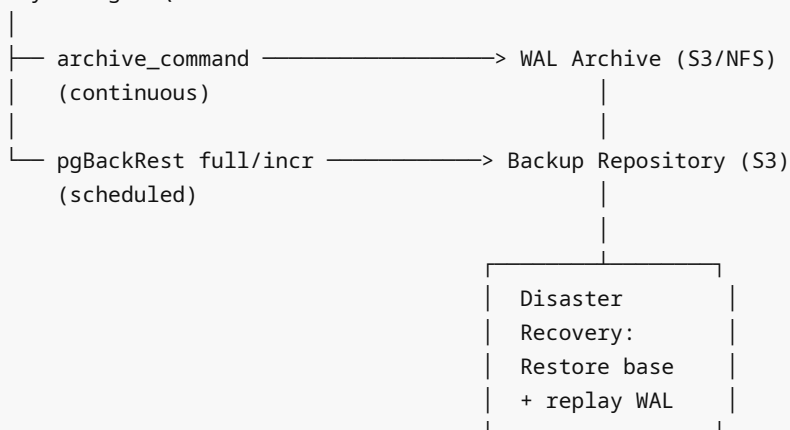
Monthly full backups → cold storage (S3 Glacier)

Retention: 7 years (compliance requirement)

RPO/RT0: Compliance recovery only (hours)

BACKUP FLOW DIAGRAM:

Primary PostgreSQL



Common Mistakes

1. **Backup without testing restore** — most common; backup is useless if restore fails
2. **Only keeping one backup copy** — hardware failure can destroy backup and source
3. **Not monitoring backup success/failure** — assuming backups are running

4. **Backup to same server as data** — server failure loses both
 5. **No encryption on backup files** — sensitive data exposed in backup storage
 6. **Not including pg_global** with pg_dump — global objects (roles) not backed up
 7. **WAL archiving not configured** — no PITR capability
 8. **Backup during peak hours** — performance impact on production
 9. **Not versioning backup scripts** — unclear what was backed up
 10. **Not documenting RTO/RPO** — no way to know if requirements are met
-

Best Practices

1. **Follow 3-2-1 rule**: 3 copies, 2 media types, 1 offsite
 2. **Test restores regularly** — monthly for production
 3. **Monitor backup jobs** — alert on failure within minutes
 4. **Encrypt backup files** — even at rest in S3
 5. **Backup from standby** — reduce primary load
 6. **Define RPO/RTO** before choosing backup strategy
 7. **Automate backup verification** — verify backup can be restored
 8. **Keep backup logs** — timestamp, size, duration, success/failure
 9. **Store backups cross-region** — protect against regional outages
 10. **Document recovery procedures** — runbook for on-call team
-

Interview Questions

Q1: What is the difference between RPO and RTO?

A: RPO (Recovery Point Objective) is the maximum acceptable age of data at time of recovery — how much data you can afford to lose. RTO (Recovery Time Objective) is the maximum acceptable time from failure to recovery — how long the system can be down. They drive backup strategy: RPO → determines backup frequency and WAL archiving; RTO → determines restore speed and affects choice of physical vs. logical backup.

Q2: Why do you need both HA (replication) and backups?

A: They solve different problems. Replication copies data in near-real-time and handles server failures (hardware crash, OS crash). It does NOT protect against logical errors: `DROP TABLE`, wrong `UPDATE`, or corruption that propagates to the replica immediately. Backups provide a snapshot in time — you can restore to before the error occurred. You need both for complete data protection.

Q3: What is the difference between logical and physical backups?

A: Logical backups export database objects as SQL (pg_dump). They're portable across versions/platforms, selective, but slow for large databases and don't support PITR. Physical backups copy raw data files (pg_basebackup/pgBackRest). They're fast, support PITR with WAL archiving, but require same major PostgreSQL version for restore and copy everything (bloat included).

Q4: What is a differential vs. incremental backup?

A: Differential: backs up everything changed since the last FULL backup. Each differential grows through the week. Restore needs 2 files: full + latest differential. Incremental: backs up only changes since the last backup (full or any incremental). Smallest individual backups, but restore requires full + all subsequent incrementals in sequence.

Q5: What is WAL archiving and why is it needed for PITR?

A: WAL (Write-Ahead Log) records every change to the database. WAL archiving copies WAL segments to a safe location as they're generated. Combined with a base backup, you can restore to any point in time by: (1) restore base backup, (2) replay WAL up to the desired timestamp. Without WAL archiving, you can only restore to the exact time of the base backup.

Q6: What is the 3-2-1 backup rule?

A: 3 copies of data (original + 2 backups), on 2 different media types (e.g., local disk + cloud), with 1 copy offsite. This protects against: single device failure (3 copies), media type failure (2 media), site disaster (1 offsite). For PostgreSQL: primary data (original), local backup (media 1), S3/cloud backup offsite (media 2, offsite).

Q7: When should you use pg_dump vs pg_basebackup?

A: `pg_dump` is preferred for: small databases (<50GB), cross-version migration, single-database exports, selective table restores, readable backup format. `pg_basebackup` is preferred for: large databases, setting up streaming replicas, when PITR is needed (combined with WAL archiving), when restore speed matters. For production, use pgBackRest which builds on physical backups with full PITR support.

Exercises and Solutions

Exercise 1: Design Backup Policy

Given: E-commerce database, 500GB, RPO = 15 min, RTO = 1 hour.

Backup Policy Design:

Tool: pgBackRest with S3

Schedule:

- Full backup: Sunday 02:00 UTC (off-peak, from standby)
- Differential: Mon-Sat 02:00 UTC
- WAL archiving: Continuous (every 5min or 16MB)

Retention:

- Full: 4 weeks
- Differential: 1 week
- WAL: 4 weeks

Storage:

- Primary: S3 us-east-1 (production region)
- Secondary copy: S3 us-west-2 (DR region) via cross-region replication

RPO verification:

- WAL archive lag monitored < 5 min (alert at 10 min)
- Target RPO: 15 min = achievable with continuous WAL archiving

RTO verification:

- Monthly restore test from S3 to test server
 - Document restore time for current DB size
 - 500GB typically: 45-75 min with pgBackRest parallel restore
-

Cross-References

- [02_pg_dump_restore.md](#) — Logical backup details
- [03_pg_basebackup.md](#) — Physical backup details
- [04_pitr.md](#) — Point-in-time recovery
- [05_wal_archiving.md](#) — WAL archiving setup
- [06_pgbackrest.md](#) — pgBackRest production setup
- [07_disaster_recovery.md](#) — DR planning